

Presents:

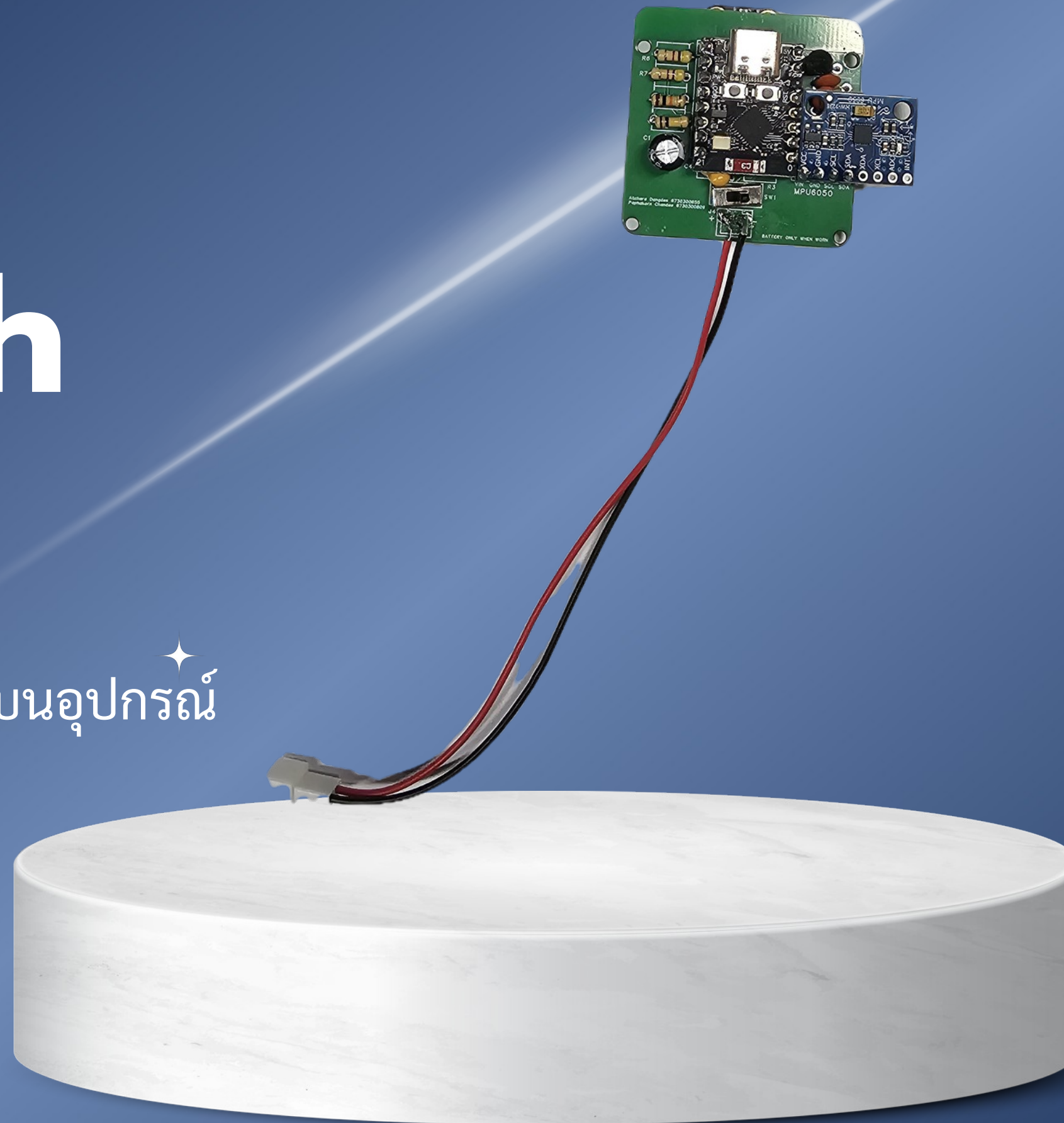
Wireless- Polygraph

2026

เครื่องจับเท็จไร้สายแบบสวมข้อมือ

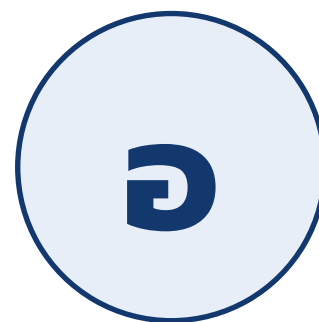
ESP32-C3 · FreeRTOS · WiFi · AI บนอุปกรณ์

03603323 Introduction to Embedded Systems
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
มหาวิทยาลัยเกษตรศาสตร์ วิทยาเขตศรีราชา



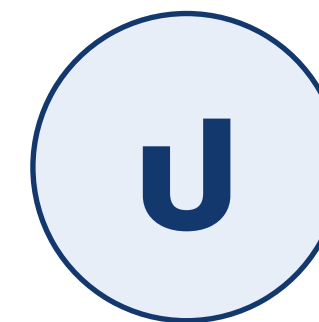
สมาชิก

Members



อัจฉรา ดังดี

6730300655



ปภากร จันทรดี

6730300809

วิศวกรรมคอมพิวเตอร์ มก. ศรีราชา · ภาคต้น ปีการศึกษา 2569

Agenda

01

Overview

ที่มา เป้าหมาย และภาพรวมระบบ

02

Hardware Design

Schematic, PCB, เซนเซอร์ 4 ตัว, แบตเตอรี่, ฐานยึด

03

Firmware & Embedded

FreeRTOS, I2C/ADC, หน่วยความจำ, Watchdog, Sleep

04

Software & UI

หน้าเว็บในนาฬิกา, Polygraph Studio บนคอม/มือถือ

05

AI & Backend

เก็บข้อมูลลง CSV, Label จริง/โกหก, เทรนและใช้บนนาฬิกา

06

Limitations & Solutions

ปัญหาที่เจอ สาเหตุ และวิธีแก้

07

Live Demo

สาธิตการใช้งานจริง

01

Overview

ที่มา เป้าหมาย และภาพรวมระบบ



ทำไมถึงอยากทำโครงการนี้

ปัญหา / แรงบันดาลใจ

เครื่องจับเท็จทั่วไป ใช้สายต่อหลายเส้น ตัวเครื่องใหญ่ ต้องนั่งติดกับคอม

ร่างกายฟ้องเมื่อตื่นเต้น ชีพจรเร่ง เหงื่อออก หลอดเลือดปลายมือหด มือสั่น ผิวย่นลง

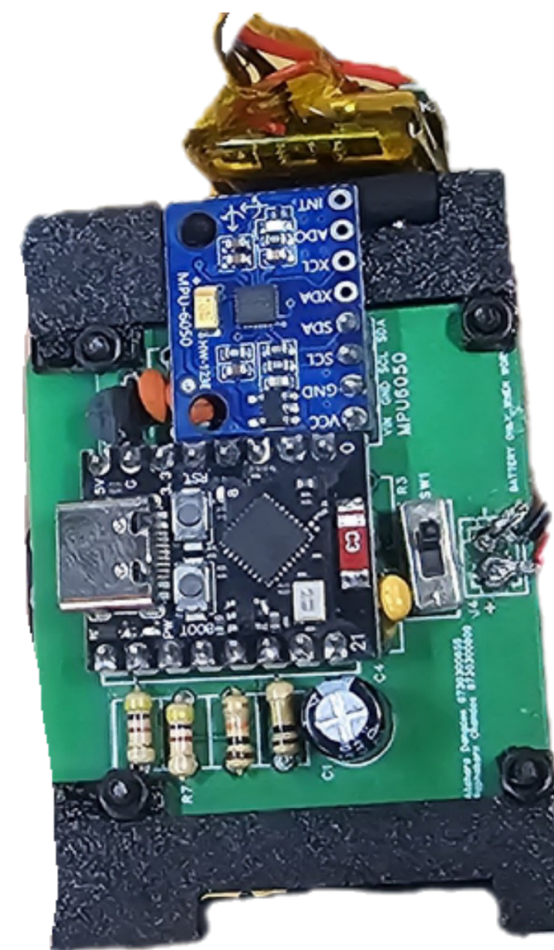
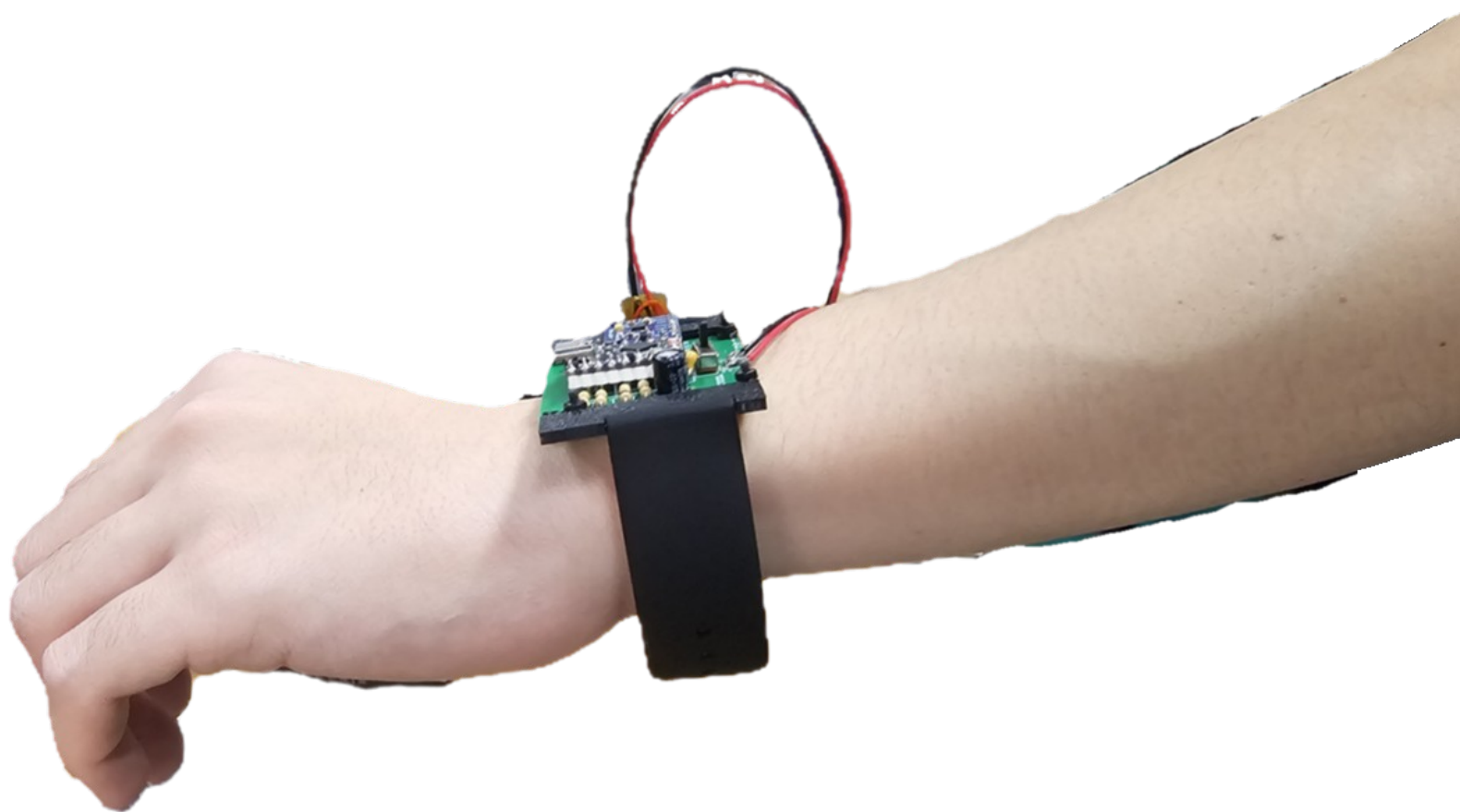
โจทย์ของเรา ทำให้เล็กพอใส่ข้อมือ ไร้สาย ดูผลได้จากมือถือ

โอกาสฝึกครบวงจร เซนเซอร์ → เฟิร์มแวร์ RTOS → WiFi → เว็บ → AI

เป้าหมาย

- 1 วัดสัญญาณร่างกาย 4 ชนิด ด้วยเซนเซอร์ที่ประกอบบน PCB ของเราเอง
- 2 ตัดสิน "จริง / โกหก" บนนาฬิกาเอง (edge computing) ไม่ต้องพึ่งคอม
- 3 ดูค่าสด กราฟ และผลได้ทั้งบนมือถือและคอมผ่าน WiFi
- 4 เก็บข้อมูลลง CSV เพื่อเทรน AI แล้วส่งโมเดลกลับเข้านาฬิกา

หน้าตาโดยรวม



4

เซนเซอร์สี่ระ

6

FreeRTOS task

5 Hz

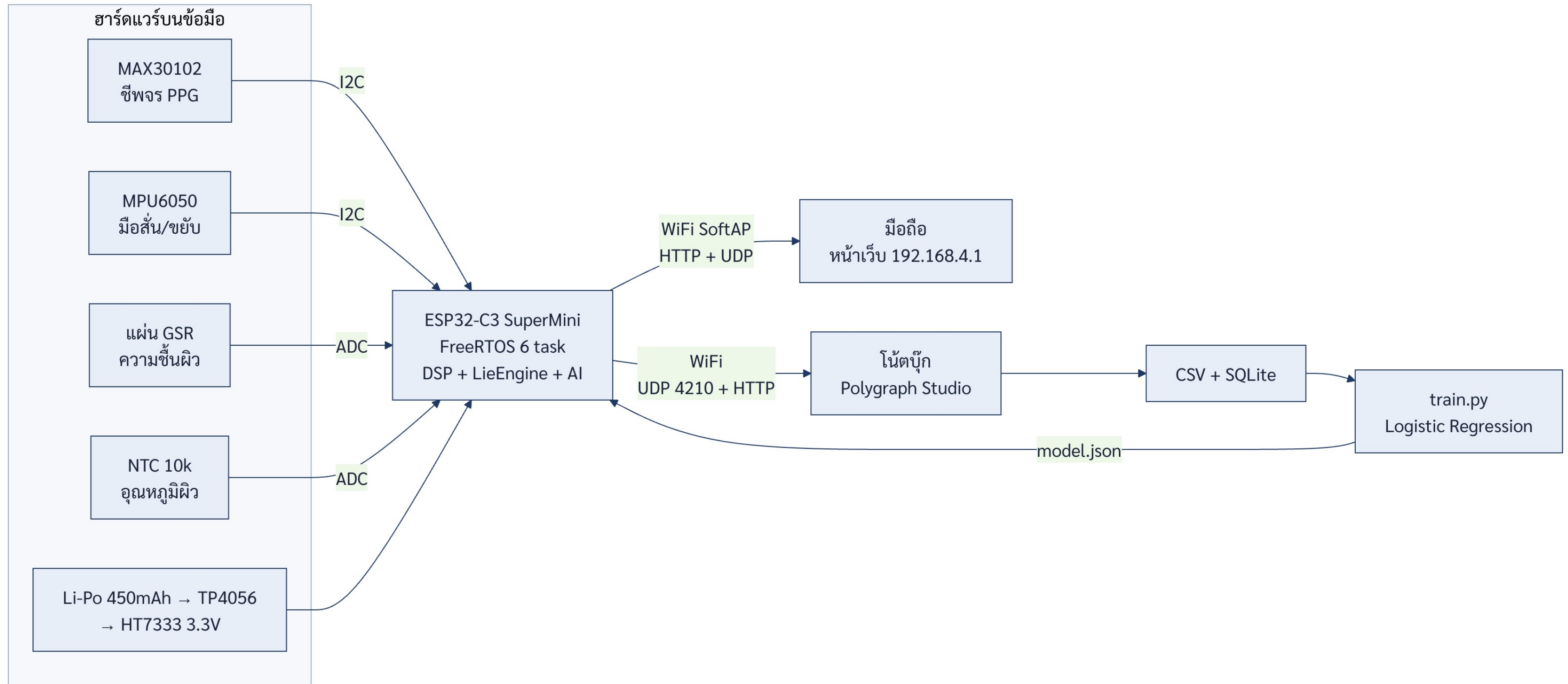
ส่งค่าสดผ่าน
WiFi

12

feature ของ AI

ESP32-C3 SuperMini สมองหลัก + WiFi ในตัว
Li-Po 3.7 V 450 mAh ชาร์จผ่าน TP4056 ใช้งานไร้สาย
ดูผลได้ 2 ทาง มือถือ (192.168.4.1) และโปรแกรมบนคอม

ภาพรวมระบบ

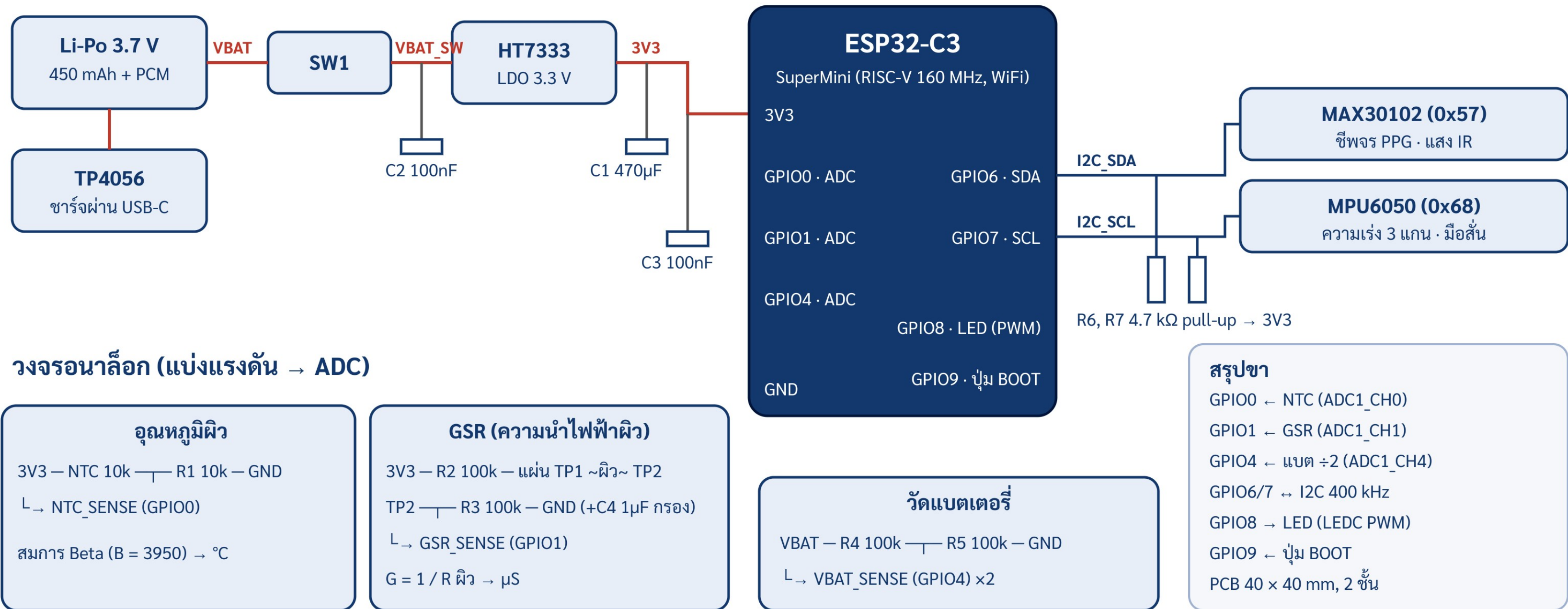


02

Hardware Design

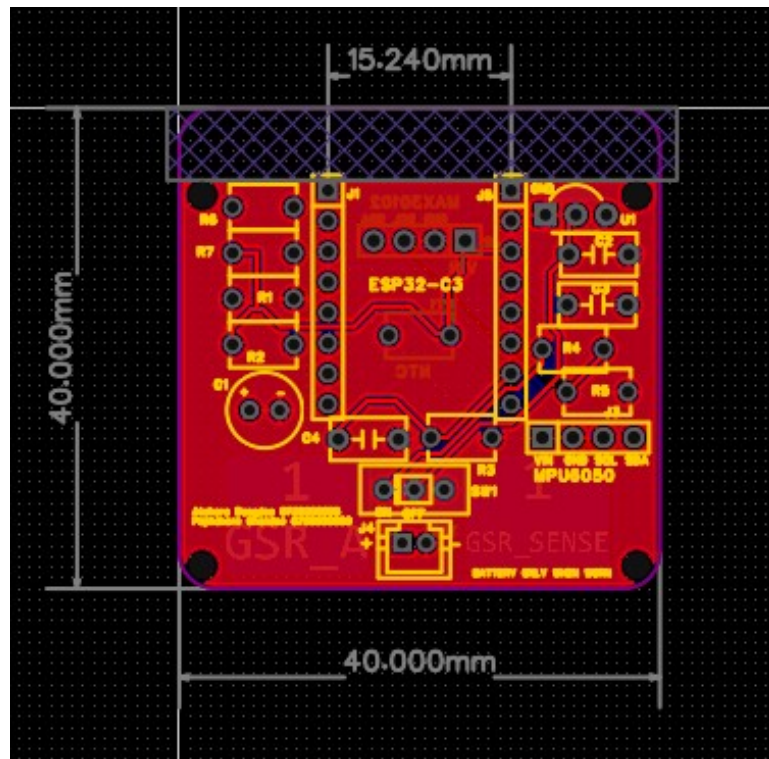
Schematic · PCB · เซนเซอร์ · แหล่งจ่ายไฟ · ฐานยึด



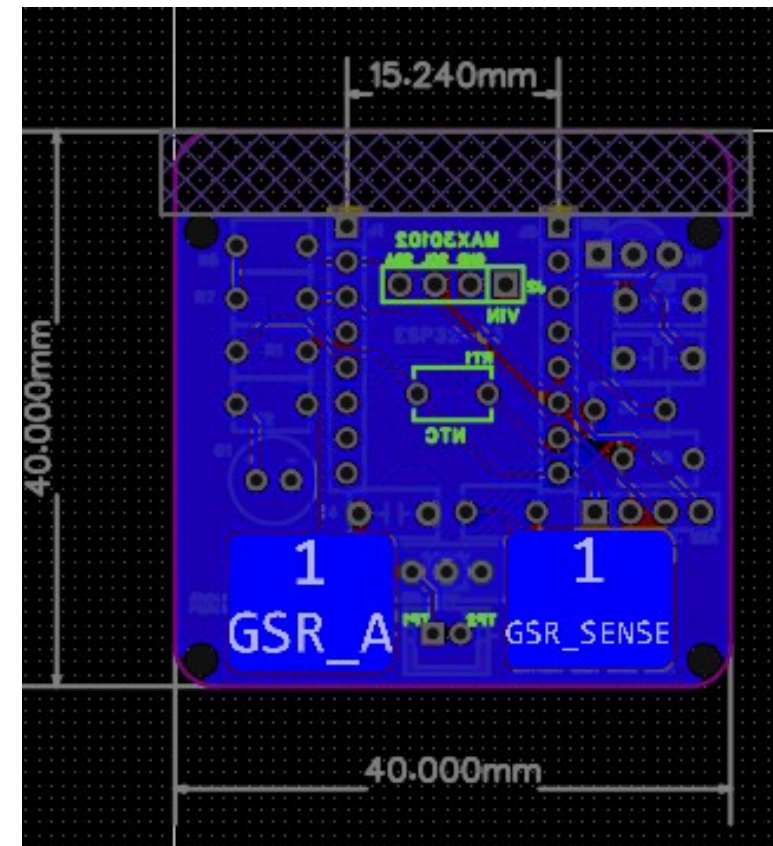


9

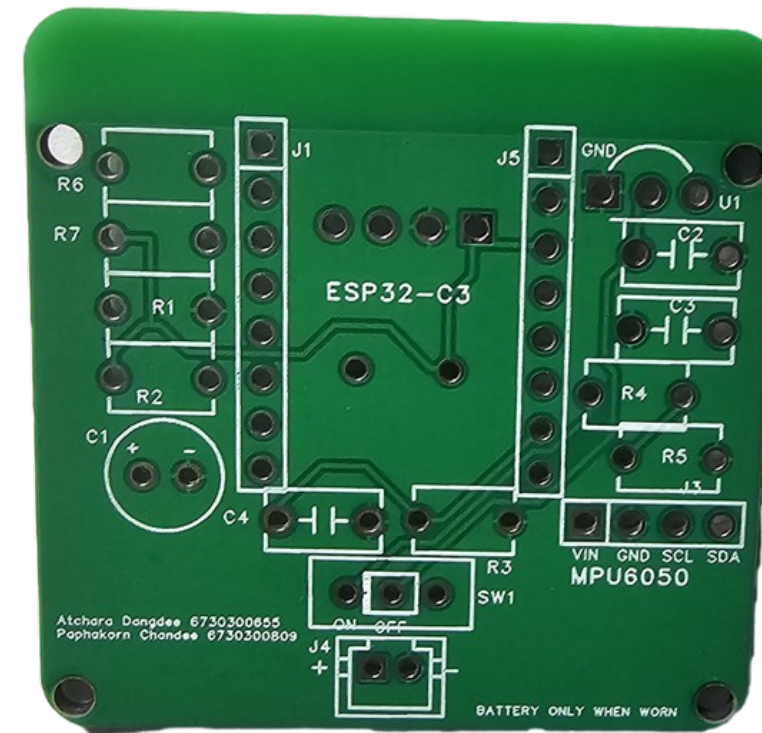
PCB



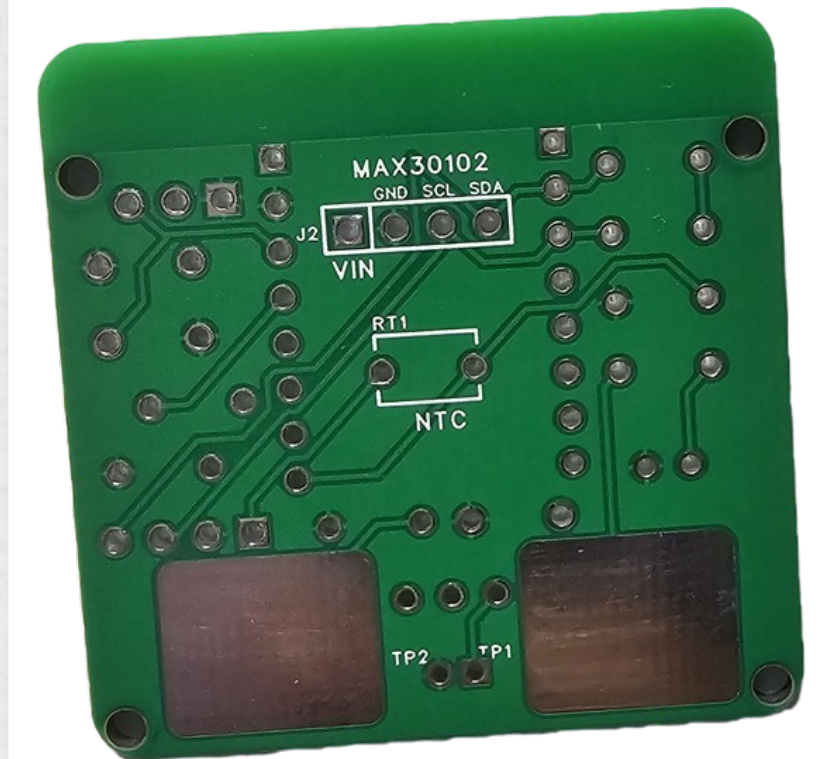
ด้านบน (Top layer)



ด้านล่าง (Bottom layer)



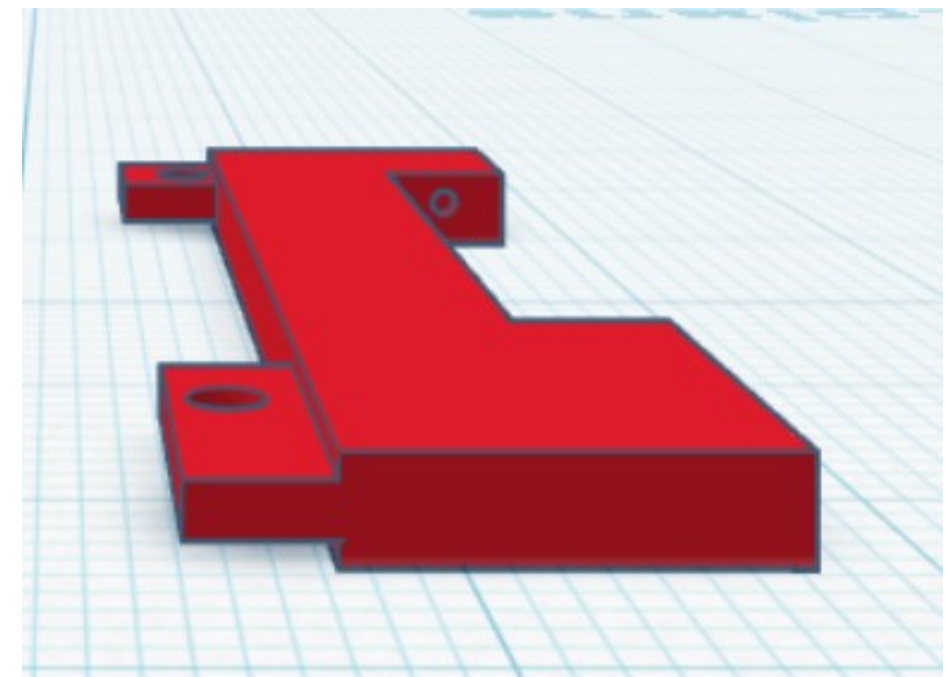
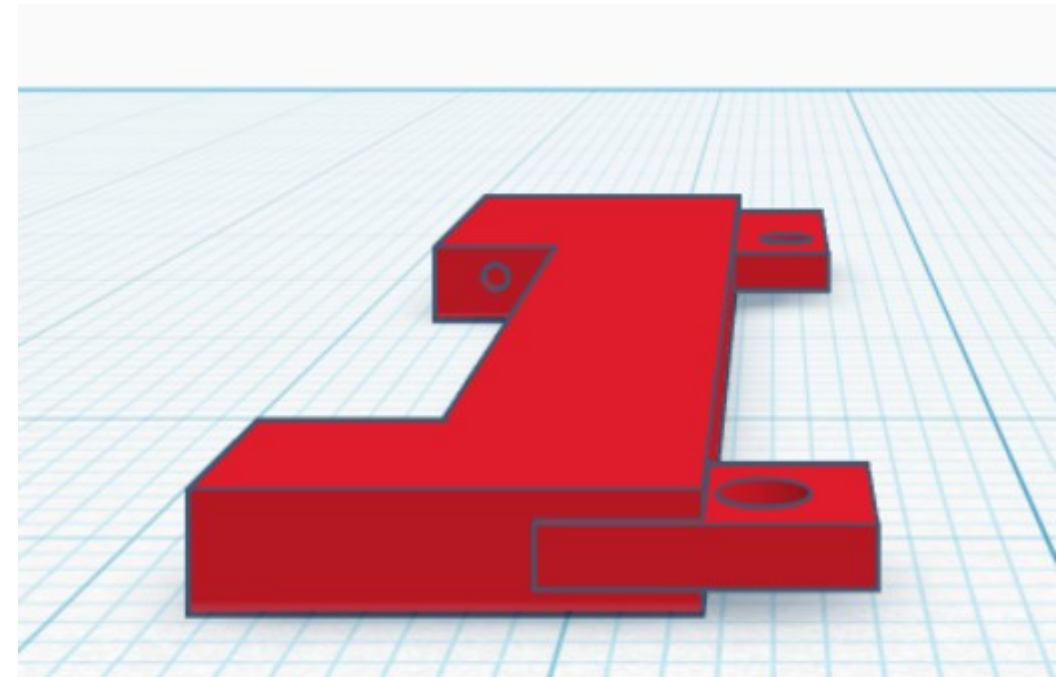
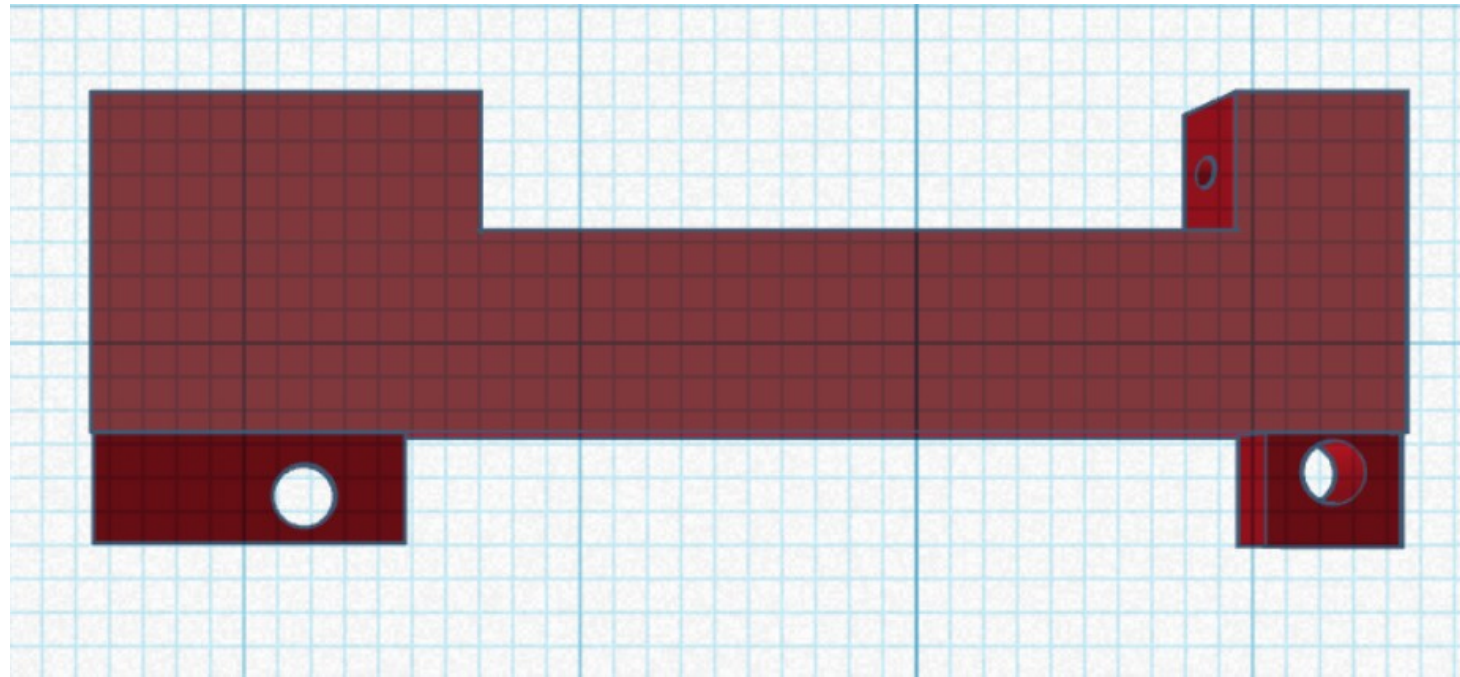
ภาพจริง ด้านบน



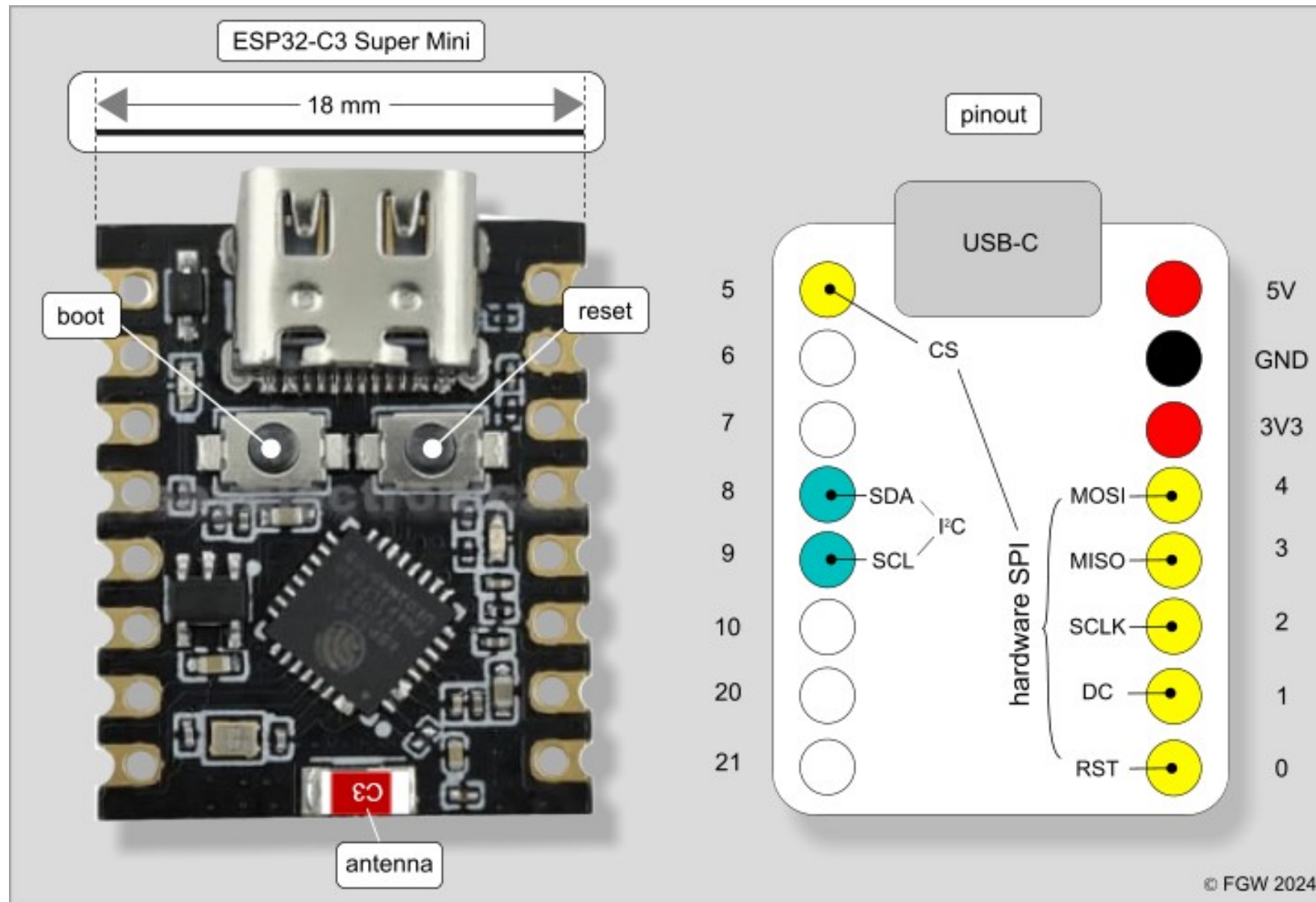
ภาพจริง ด้านล่าง — แผ่น GSR

2 ชั้น • 40 × 40 มม. วางบอร์ด ESP32-C3 บนซ็อกเก็ต ถอดเปลี่ยนได้
แผ่น GSR บน PCB ใช้ลายทองแดงเป็นขั้วสัมผัสผิว ไม่ต้องมีสายแยก

ฐานยึดสายและตัวบอร์ด



บอร์ด ESP32-C3 SuperMini



สเปก

- RISC-V 32-bit 160 MHz
- SRAM 400 KB
- RTC memory 8 KB
- Flash 4 MB
- WiFi 2.4 GHz + BLE 5

ใช้ขาไหนบ้าง

GPIO6/7 I2C (MAX30102, MPU6050)

GPIO0/1/4 ADC (NTC, GSR, แบต)

GPIO8/9 LED PWM / ปุ่ม BOOT

เซนเซอร์ 4 ชนิด



MAX30102

ชีพจร (PPG)

แสง IR สะท้อนจากเส้นเลือด →
ชีพจร, HRV, แรงชีพจร

I2C 0x57



MPU6050

การสั่น / ขยับ

ความเร่ง 3 แกน → มือสั่น 3–
15 Hz, การขยับตัว

I2C 0x68

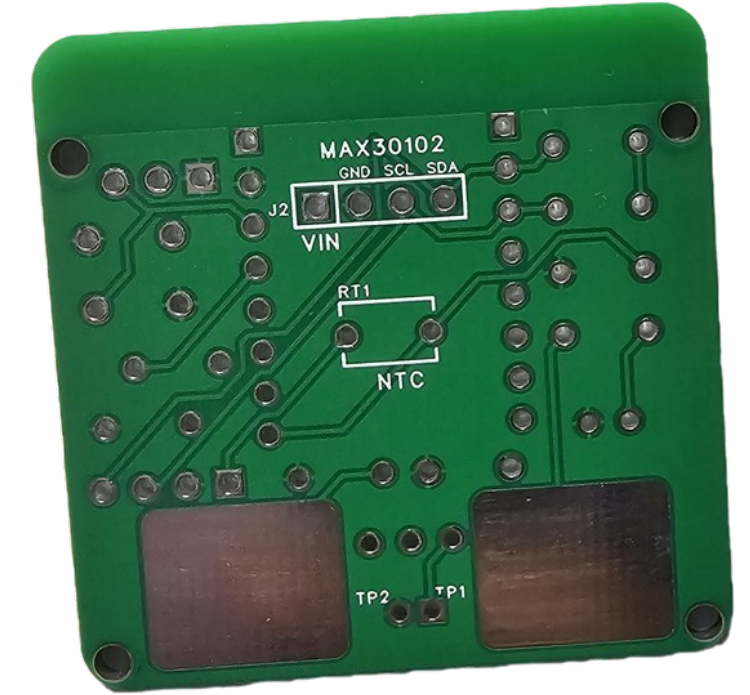


NTC 10 k Ω

อุณหภูมิผิว

ความต้านทานเปลี่ยนตาม
อุณหภูมิ → °C

ADC GPIO0



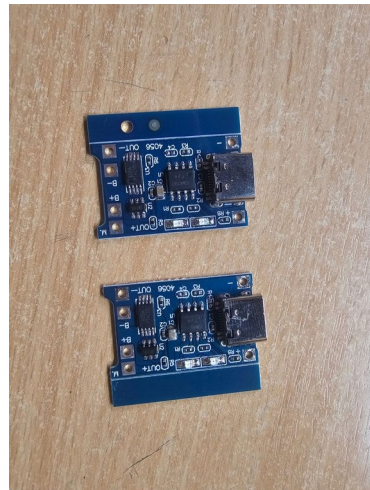
แผ่น GSR

ความชื้นผิว

เหงื่อทำให้ผิวนำไฟฟ้าดีขึ้น →
 μS

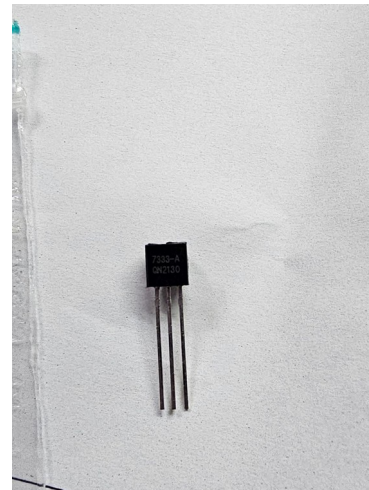
ADC GPIO1

อุปกรณ์อื่น ๆ



TP4056

ชาร์จแบตเตอรี่ Li-Po
ผ่าน USB-C



HT7333

LDO แปลงแบตเตอรี่
→ 3.3 V



สวิตช์เลื่อน

เปิด/ปิดไฟจาก
แบตเตอรี่



ตัวเก็บประจุ เซรามิก

decoupling ไกลล์
ขาชิป



R 4.7 k Ω

I2C pull-up



R 10 k Ω

คู่กับ NTC



R 100 k Ω

วงจร GSR + วัด
แบตเตอรี่



ตัวเก็บประจุ MLCC

กรองสัญญาณ
GSR และไฟเลี้ยง

ระยะเวลาใช้งานแบตเตอรี่



ทางทฤษฎี (ประมาณการ)

ส่วนประกอบ	กระแสเฉลี่ย
ESP32-C3 + WiFi SoftAP	≈ 90 mA *
MAX30102 (LED IR)	≈ 2 mA *
MPU6050	≈ 4 mA *
NTC/GSR/วัดแบตเตอรี่	< 0.2 mA
รวม	≈ 100 mA

$450 \text{ mAh} \times 80\% \text{ (ตัดที่ } 3.3 \text{ V)} \div 100 \text{ mA}$

≈ 3.6 ชั่วโมง

** ประมาณจาก datasheet และการตั้งค่าของเรา (LED IR ≈ 6.2 mA แบบพัลส์, WiFi TX 8.5 dBm) ไม่ใช่ค่าที่วัดได้*

ทางปฏิบัติ: วิธีวัดจริง

- ชาร์จเต็ม 4.2 V แล้วถอด USB
- เปิด WiFi + ส่งข้อมูลสดต่อเนื่อง (เหมือนใช้งานจริง)
- บันทึก battery_mv ทุก 0.2 วินาทีลง CSV
- จับเวลาจนแบตเตอรี่ลดถึง 3.3 V (นาฬิกาเข้า deep sleep ป้องกันแบตเตอรี่เอง)
- ลดการกินไฟ: ECO 80 MHz, ลดกำลังส่ง WiFi, หลับอัตโนมัติ

ระยะเวลาการทำงาน



ปี พ.ศ. 2569 · ช่วงก่อนสอบกลางภาคเน้นฮาร์ดแวร์ หลังสอบกลางภาคเน้นเฟิร์มแวร์ ซอฟต์แวร์ และ AI

03

Firmware & Embedded

FreeRTOS · Interrupt · I2C · ADC · Memory · Watchdog ·
Sleep



เทคนิค Embedded ที่ใช้ในโครงการ

1 GPIO + PWM

LED บอกสถานะ (LEDC 5 kHz)
· ปุ่ม BOOT + debounce

2 Interrupt + Timer

Hardware timer 100 Hz ปลุก
sensorTask จาก ISR

3 UART / USB Serial

Serial console พิมพ์คำสั่ง
status · i2c · mode

4 I2C

MAX30102 + MPU6050 ที่
400 kHz · ภูบัสค้างอัตโนมัติ

5 ADC

GSR · NTC · แบต — ADC1,
เฉลี่ย 16 ครั้ง

6 Memory

Flash map 4 MB · NVS ·
EEPROM · RTC · LittleFS

7 WiFi

SoftAP + UDP ค่าสด 5 Hz +
HTTP REST API

8 Watchdog

3 ชั้น + กล้องดำบอกสาเหตุ
รีเซ็ต

9 Low power

ECO 80 MHz · Light / Deep
sleep · Standby

10 RTOS

6 task · queue · mutex ·
event group

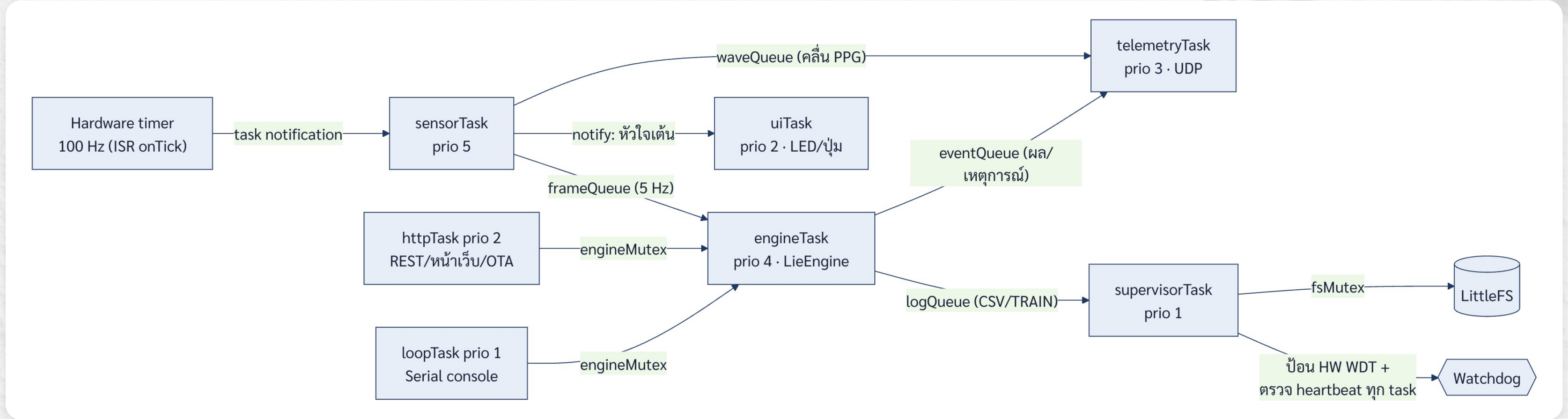
11 PCB + 3D

PCB 2 ชั้น 40×40 มม. · ฐานยึด
3 มิติ

12 Intelligence

Logistic Regression คำนวณ
บนนาฬิกาเอง

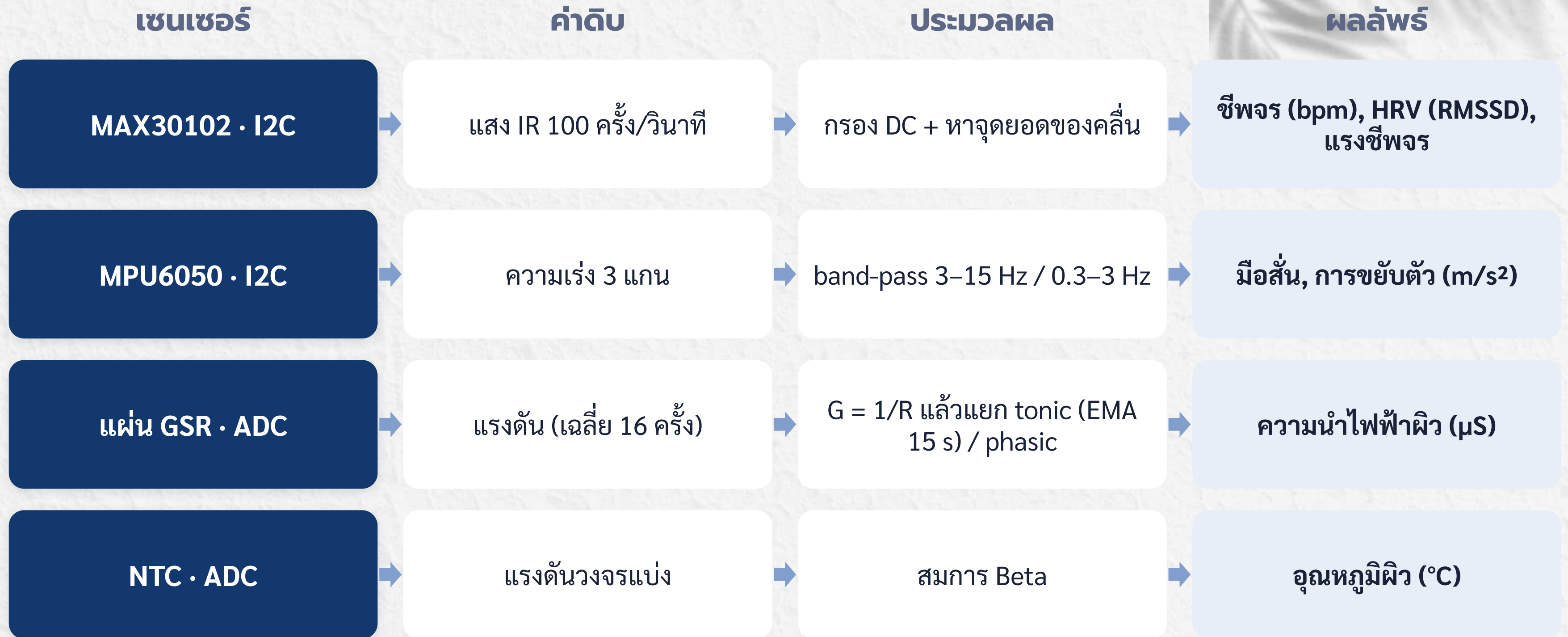
FreeRTOS: แบ่งงานเป็น 6 task



task	priority	หน้าที่
sensorTask	5	อ่านเซนเซอร์ทุก 10 ms (ถูกปลุกจาก timer interrupt)
engineTask	4	LieEngine + โมเดล AI ตัดสินจริง/โกหก 5 ครั้ง/วินาที
telemetryTask	3	ส่งค่าสด/เหตุการณ์ทาง UDP

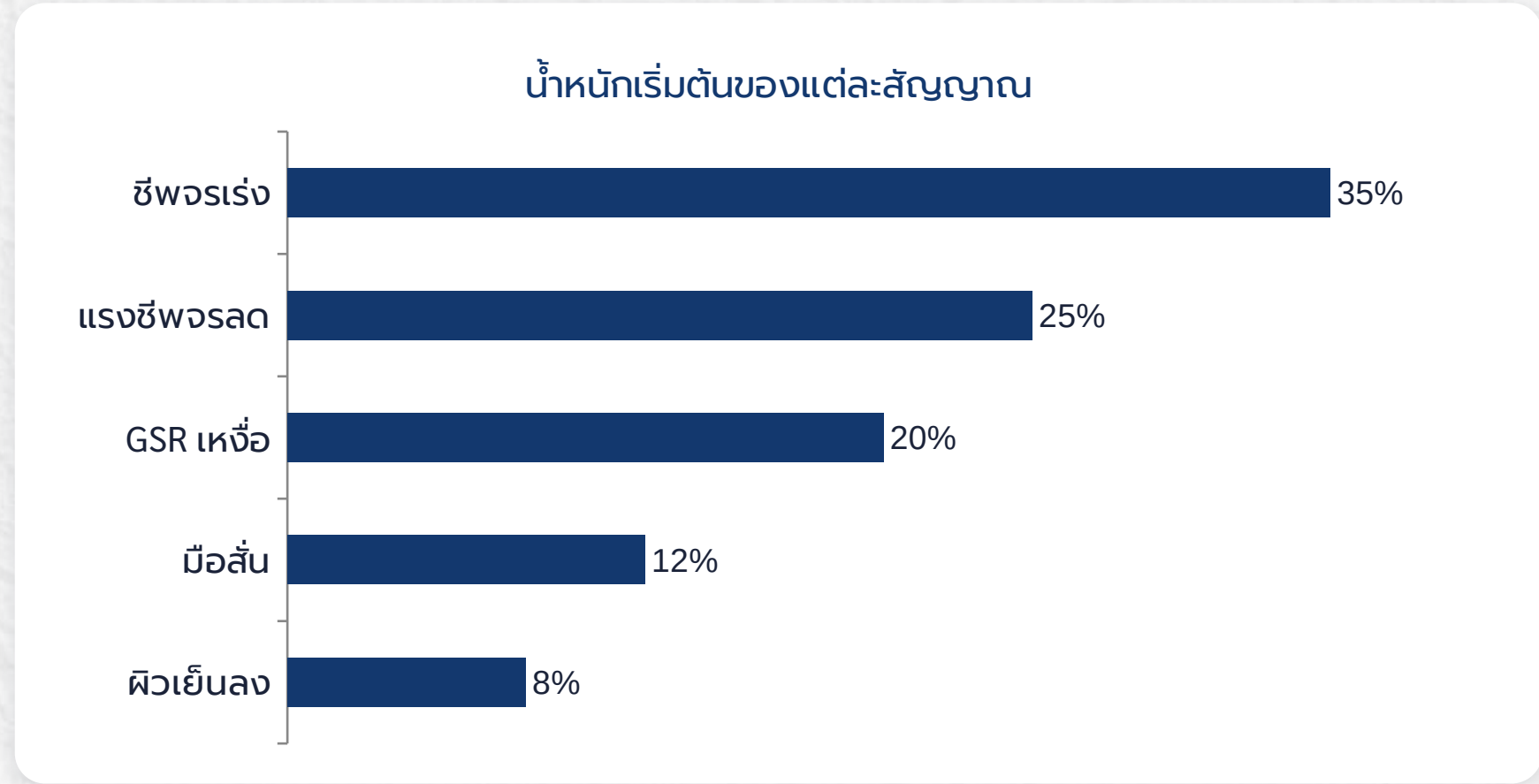
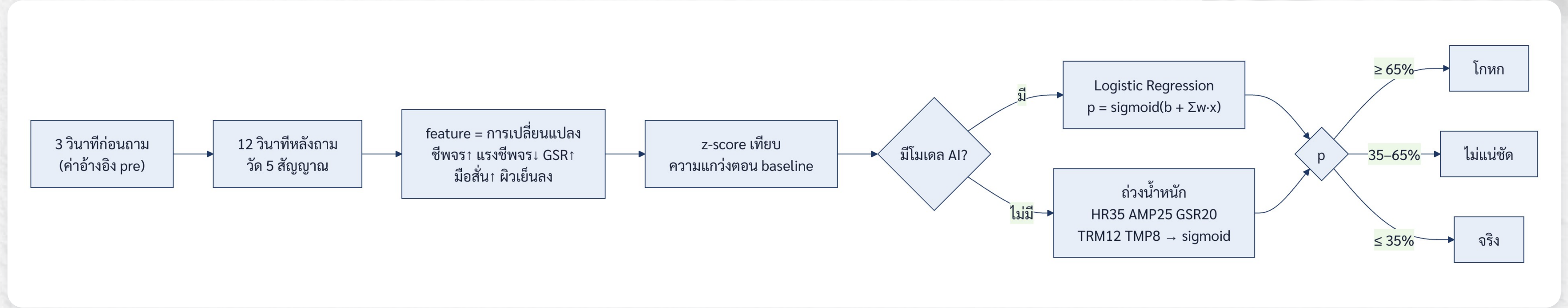
task	priority	หน้าที่
httpTask	2	หน้าเว็บ + REST API + OTA
uiTask	2	LED กระพริบตามหัวใจ + ปุ่ม
supervisorTask	1	เขียนแฟลช · เช็คแบตเตอรี่ · ป้อน watchdog

อ่านเซนเซอร์และประมวลผลสัญญาณ



รวมเป็น frame ทุก 200 ms → frameQueue → engineTask (LieEngine) และส่งทาง UDP ให้กราฟเรียลไทม์

ตัดสินใจจริง / โททกอย่างไร



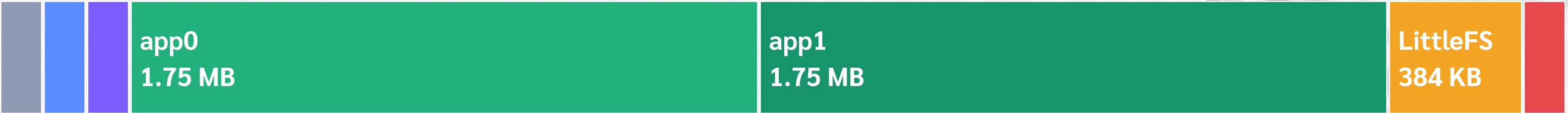
Baseline นั่งพัก 30 วินาที เพื่อรู้ "ค่าปกติ" และความกว้างของคนนั้น

z-score เปลี่ยนมากกว่าตอนพักก็เท่า — ทำให้เทียบคนต่างกันได้

ข้อควบคุม ข้อที่รู้เฉยๆ ใช้ปรับน้ำหนัก/เกณฑ์ให้เข้ากับแต่ละคน

ไม่มีเหงื่อก็ได้ GSR หาย → ใช้ 4 สัญญาณที่เหลือแทน

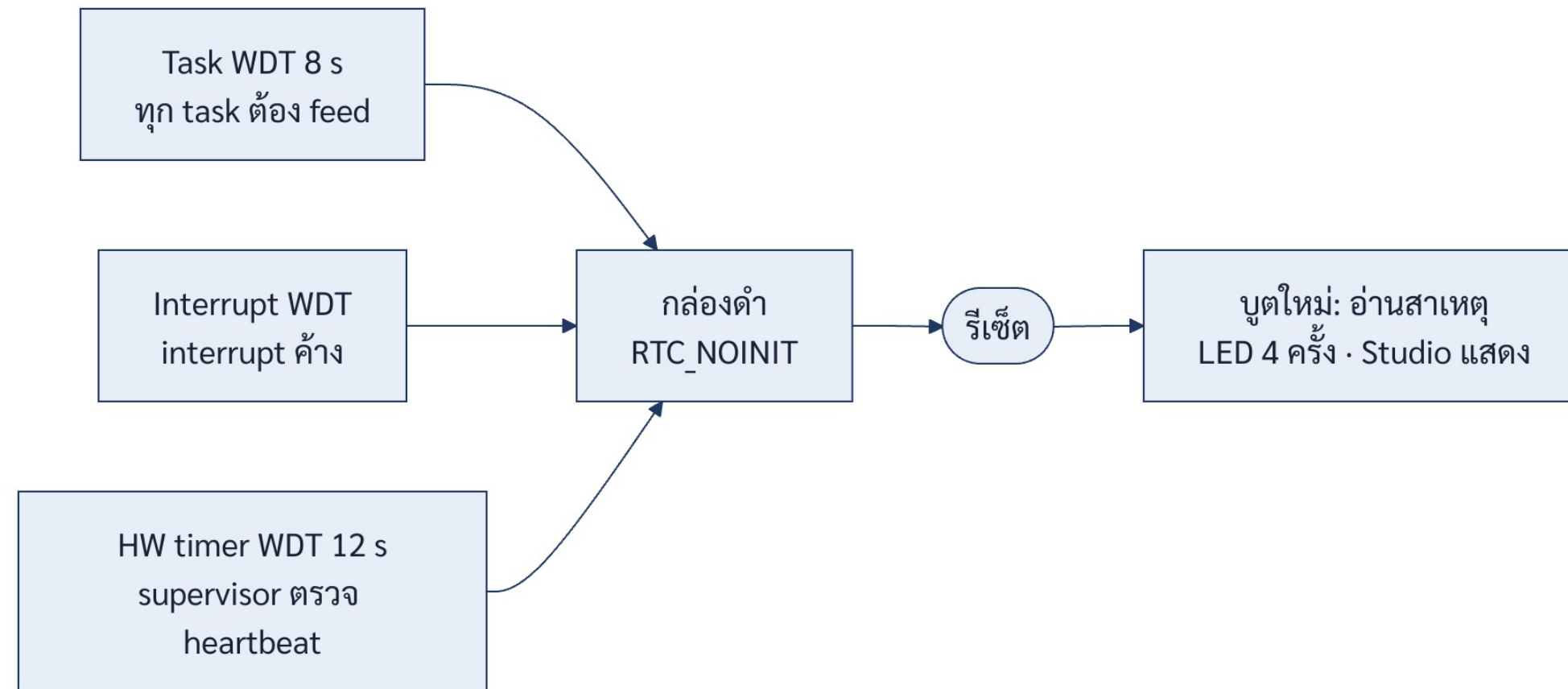
หน่วยความจำ: Flash Map 4 MB



0x0000 bootloader · 0x9000 nvs · 0xE000 otadata · 0x10000 app0 · 0x1D0000 app1 · 0x390000 LittleFS · 0x3F0000 coredump

ที่เก็บ	อยู่ที่	เก็บอะไร	ทำอะไรได้
NVS (Preferences)	Flash	ค่าตั้ง, โหมด, โมเดล AI (+ CRC32)	ไฟดับ · wear leveling ในตัว
EEPROM emulation	Flash (NVS blob)	สถิติสะสม: บุต, watchdog, ไฟตก	ไฟดับ (ต้อง commit)
LittleFS	Flash 384 KB	events.log, results.csv, train.csv	ไฟดับ · ดาวน์โหลดผ่านเว็บได้
RTC_DATA_ATTR	RTC memory 8 KB	ตัวนับการตื่นจาก deep sleep	deep sleep
RTC_NOINIT_ATTR	RTC memory	"กล่องดำ" เหตุผลการรีเซ็ต	WDT · panic · restart
SRAM (stack/heap)	SRAM 400 KB	ตัวแปร, คิว, stack ของ 6 task	หายเมื่อรีเซ็ต

Watchdog 3 ชั้น + กล้องดำ



Task WDT · 8 วินาที

1

ทุก task ต้องเรียก feed() — task ไหนค้างรู้ทันที

Interrupt WDT

2

interrupt ค้าง/ปิด interrupt นานเกิน → รีเซ็ต

Hardware timer WDT · 12 วินาที

3

supervisor ตรวจ heartbeat ทุก task แล้วค่อยบ๊อง

หลังรีเซ็ตเกิดอะไรขึ้น

- อ่านกล่องดำใน RTC_NOINIT + core dump → รู้สาเหตุและชื่อ task ที่ค้าง
- ไฟ LED กระพริบ 4 ครั้งตอนบูต, Studio แจ้งเตือน, คำสั่ง boots แสดงประวัติ 8 ครั้งล่าสุด
- สาธิตได้จากหน้า "ระบบ & อุปกรณ์" หรือ Serial: wdt twdt confirm / hwwdt / panic / intwtdt

พลังงานและ Sleep Mode

โหมด	CPU	WiFi	หน่วยความจำ	ปลุกด้วย	ใช้เมื่อ
Normal	160 MHz	เปิด	อยู่ครบ	-	ใช้งานปกติ
ECO	80 MHz	เปิด	อยู่ครบ	-	ประหยัดไฟ ส่งข้อมูลช้าลง
Light sleep	หยุด	ปิด	อยู่ครบ (baseline ไม่หาย)	ปุ่ม BOOT / timer	พักชั่วคราว
Deep sleep	ปิด	ปิด	เหลือแค่ RTC memory	timer → บุตใหม่	เลิกใช้ / แบตต่ำ < 3.3 V
Standby	ปิด	ปิด	RTC memory	ตื่นเช็คทุก 20 วินาที	รอคนมาใส่نافิกา

ตั้งค่าง่าย ๆ

หน้าเว็บ เลือก "หลับอัตโนมัติ" ปิด / 5 / 10 / 30 / 60 นาที
Serial sleep off · sleep auto 10 · sleep light · sleep deep 60
ปุ่ม **BOOT** กดค้าง 5 วินาที = light sleep, 10 วินาที = standby

ไฟ LED บอกสถานะ (ไม่ต้องมีจอ)

- กระพริบ 2 / 4 / 6 ครั้งตอนบูต = ปกติ / watchdog / ไฟตก
- ติดค้าง 0.6 วินาที = เปิด WiFi สำเร็จ
- กระพริบตามจังหวะหัวใจ = วัดชีพจรอยู่
- รัว 3 วินาที = ผลโกหก · ติดค้าง = ผลจริง

04

Software & UI

หน้าเว็บในนาฬิกา · Polygraph Studio · มือถือ /
คอมพิวเตอร์



Frontend / Backend 2 ชั้น

ชั้นที่ 1 — ในนาฬิกา

Backend REST API ใน ESP32 (web_server.cpp)

Frontend หน้าเว็บ HTML/JS ฝังในแฟลช

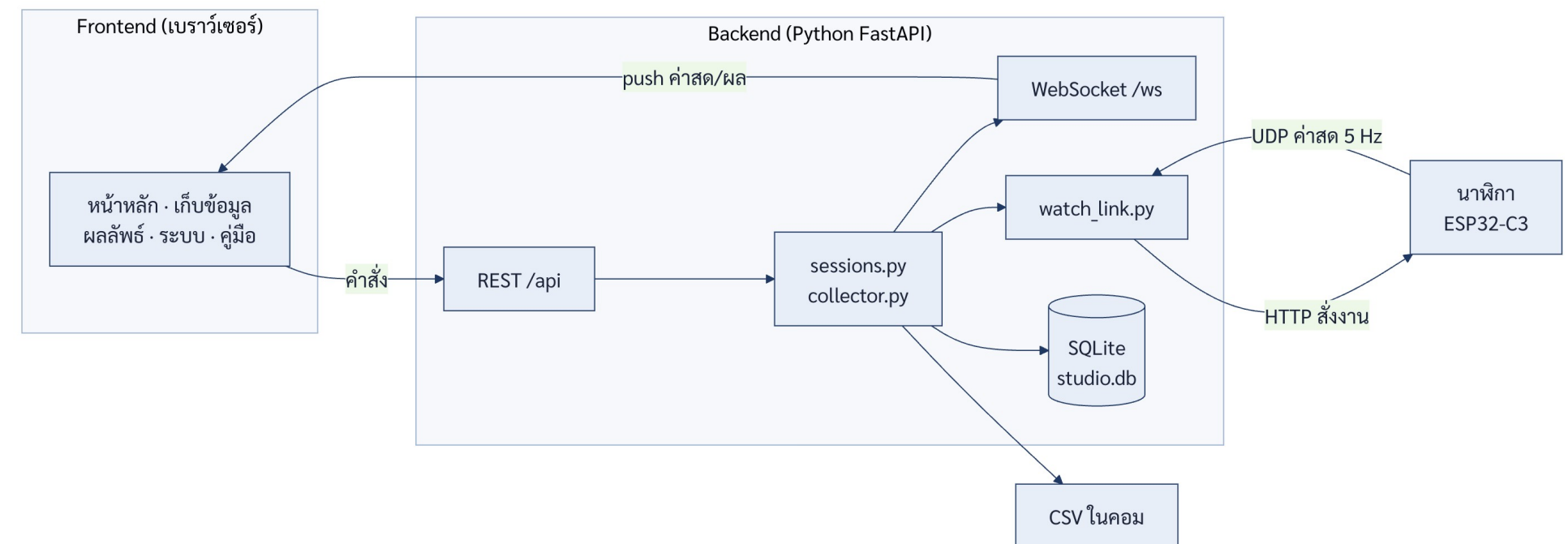
เปิดที่ <http://192.168.4.1> บนมือถือ

ชั้นที่ 2 — บนคอม (Studio)

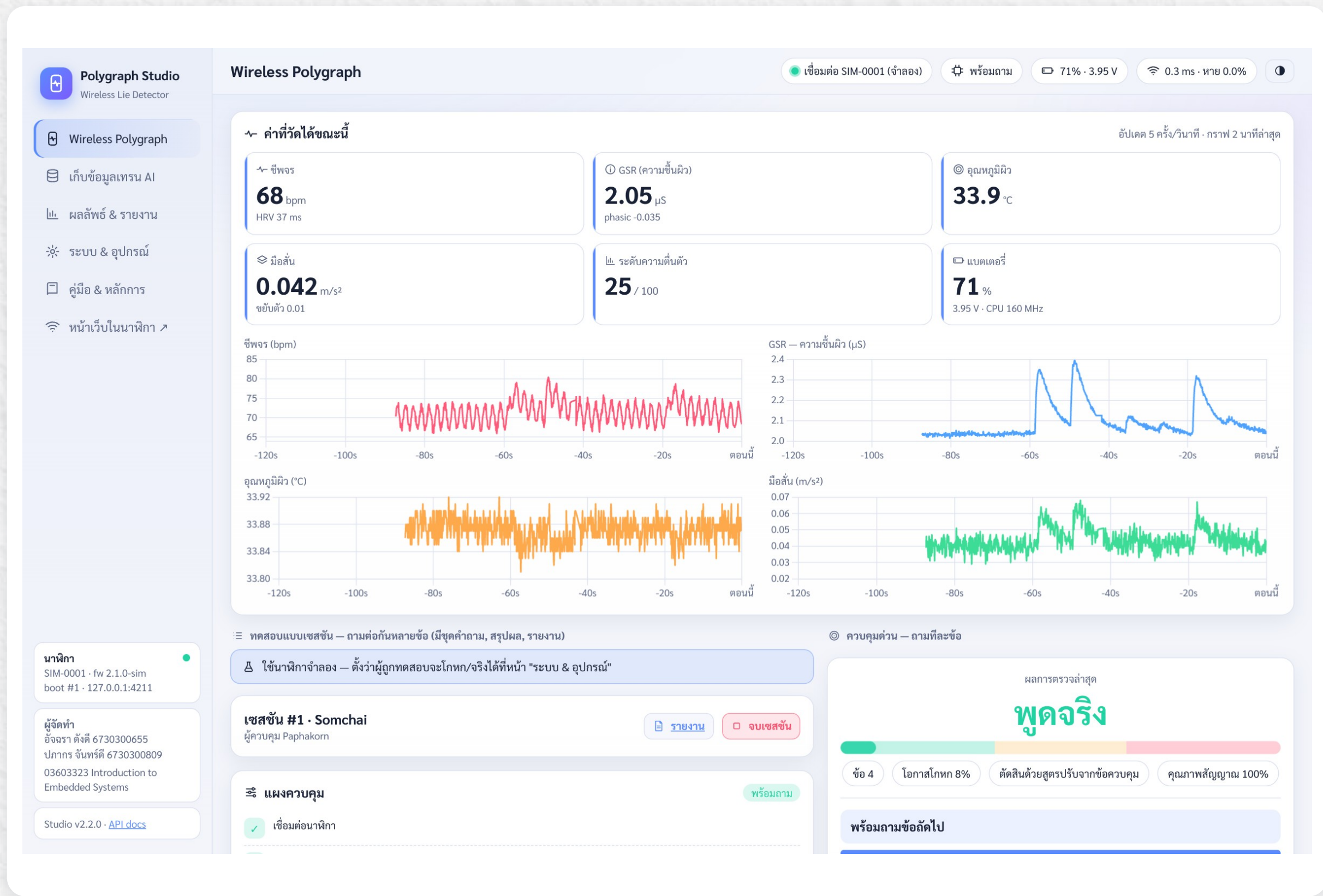
Backend Python FastAPI + SQLite + WebSocket

Frontend หน้าเว็บ 5 หน้า + กราฟเรียลไทม์

เปิดที่ `run_studio.bat` → `127.0.0.1:8000`



หน้าเว็บบน Desktop — Polygraph Studio



ค่าสด 5 ครั้ง/วินาที ชีพจร GSR อุณหภูมิ มือสั่น ความตื่นตัว แบต กราฟ 2 นาทีล่าสุด แถบสีบอกช่วง คำถาม

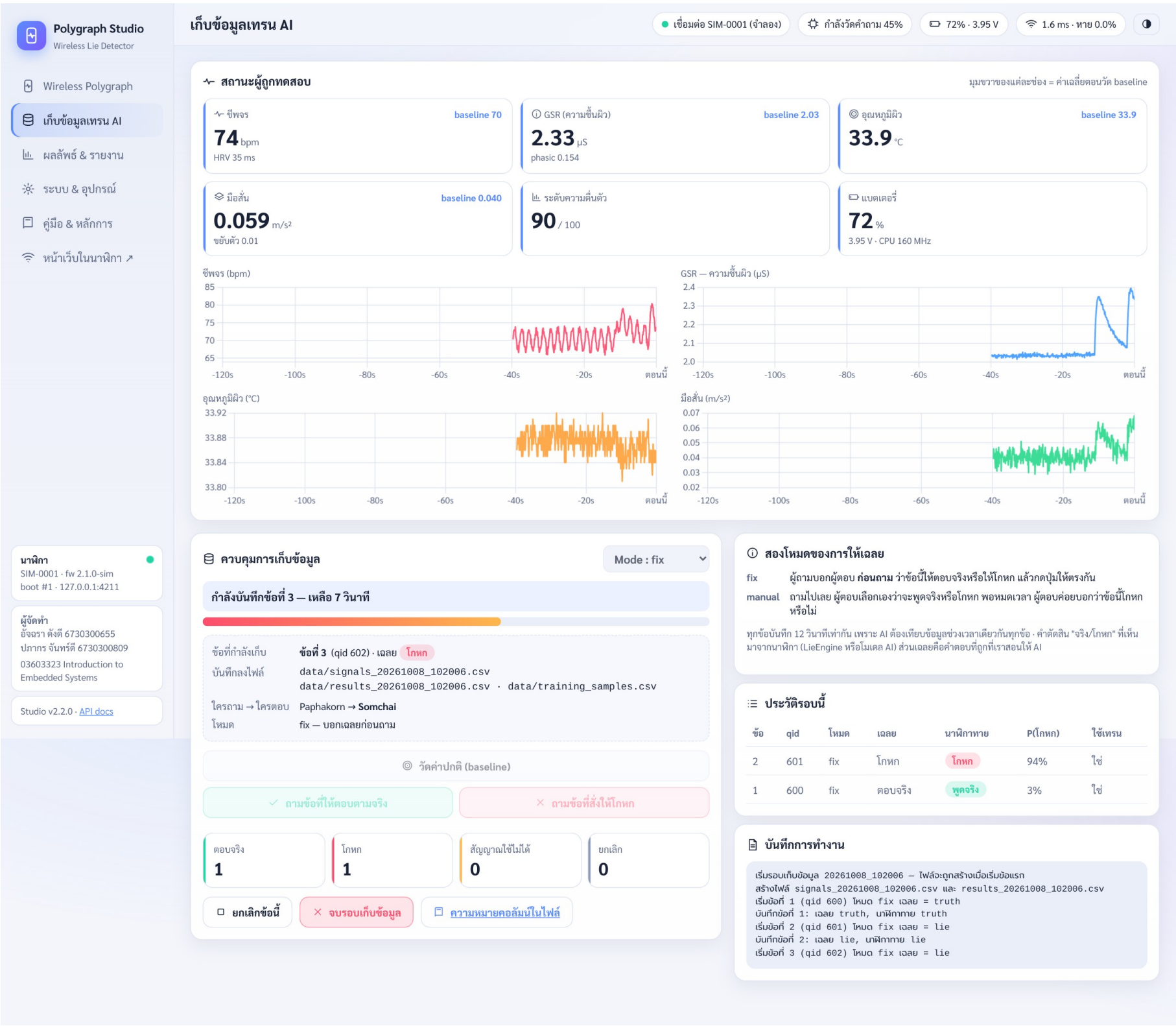
ทดสอบแบบเชสชัน ชุดคำถาม สำเร็จรูป + รายงาน PDF

ควบคุมด่วน ถามทีละข้อ ไม่ต้องสร้าง เซสชัน

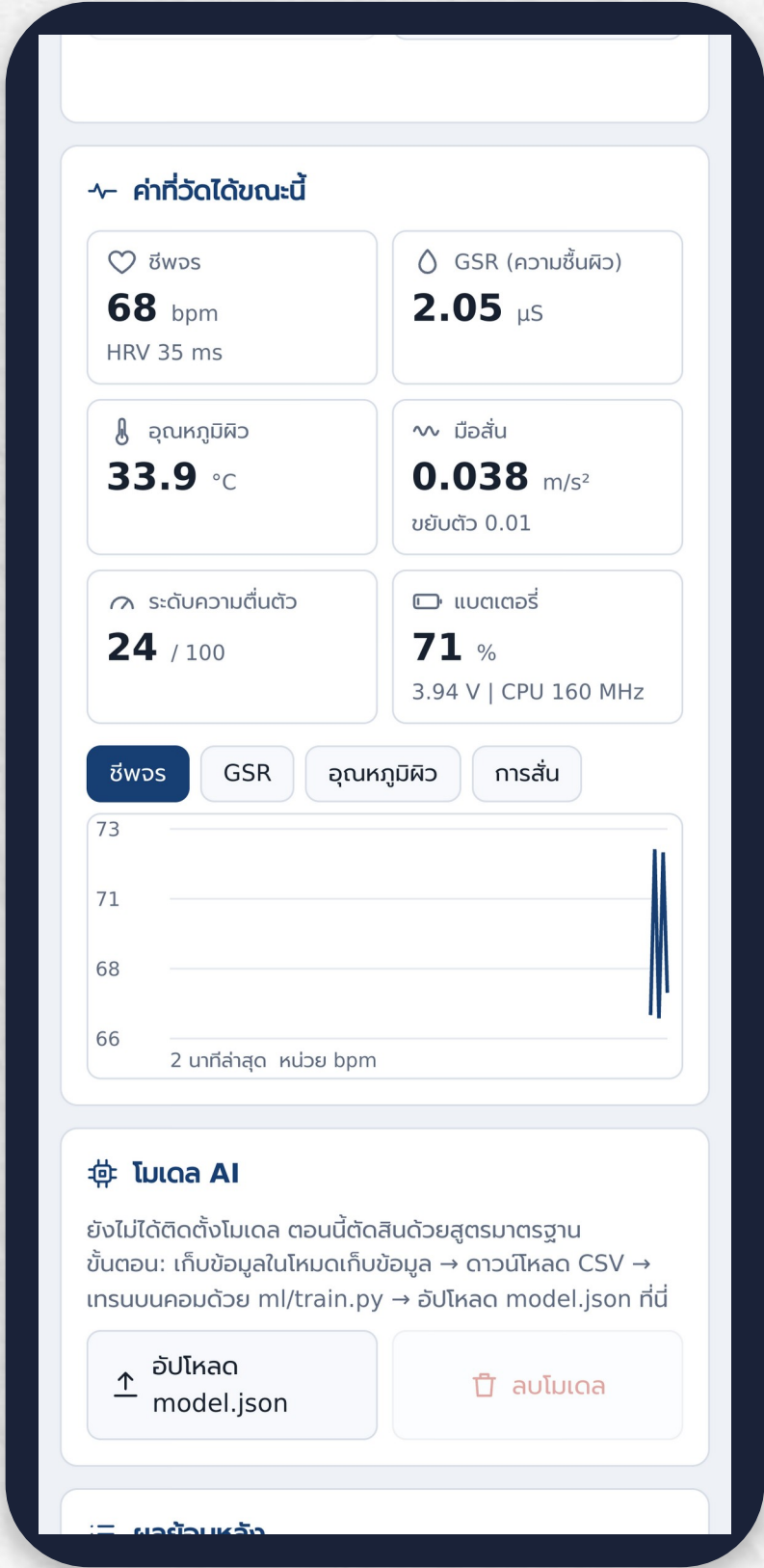
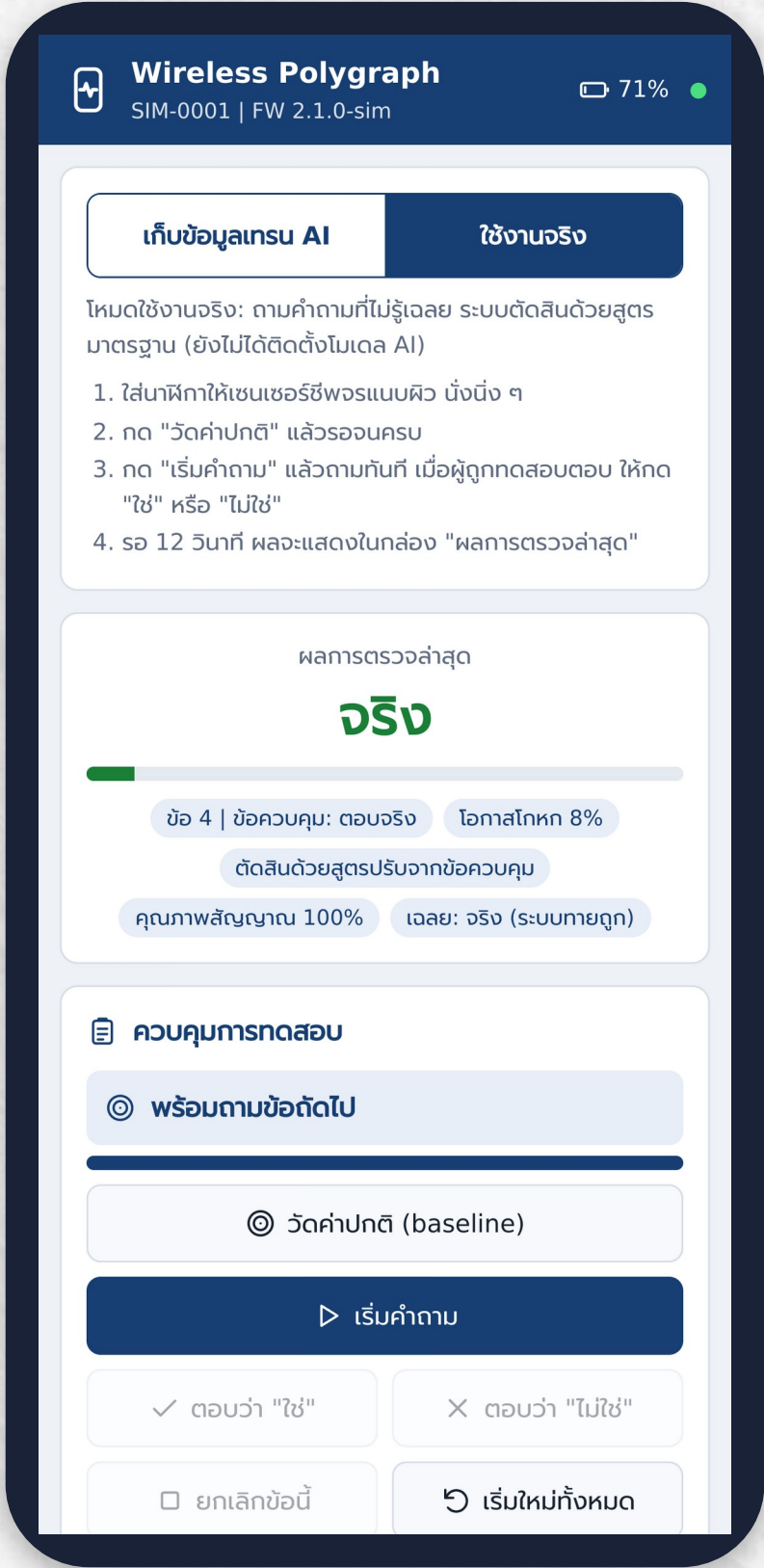
ตั้งค่า หลับอัตโนมัติ · ECO · กำลังส่ง WiFi

ภาพหน้าจอจริงของโปรแกรม (ข้อมูลจากโหมคนาฬิกาจำลองสำหรับทดสอบ)

หน้าเก็บข้อมูลเทรน AI



หน้าเว็บบนมือถือ – 192.168.4.1

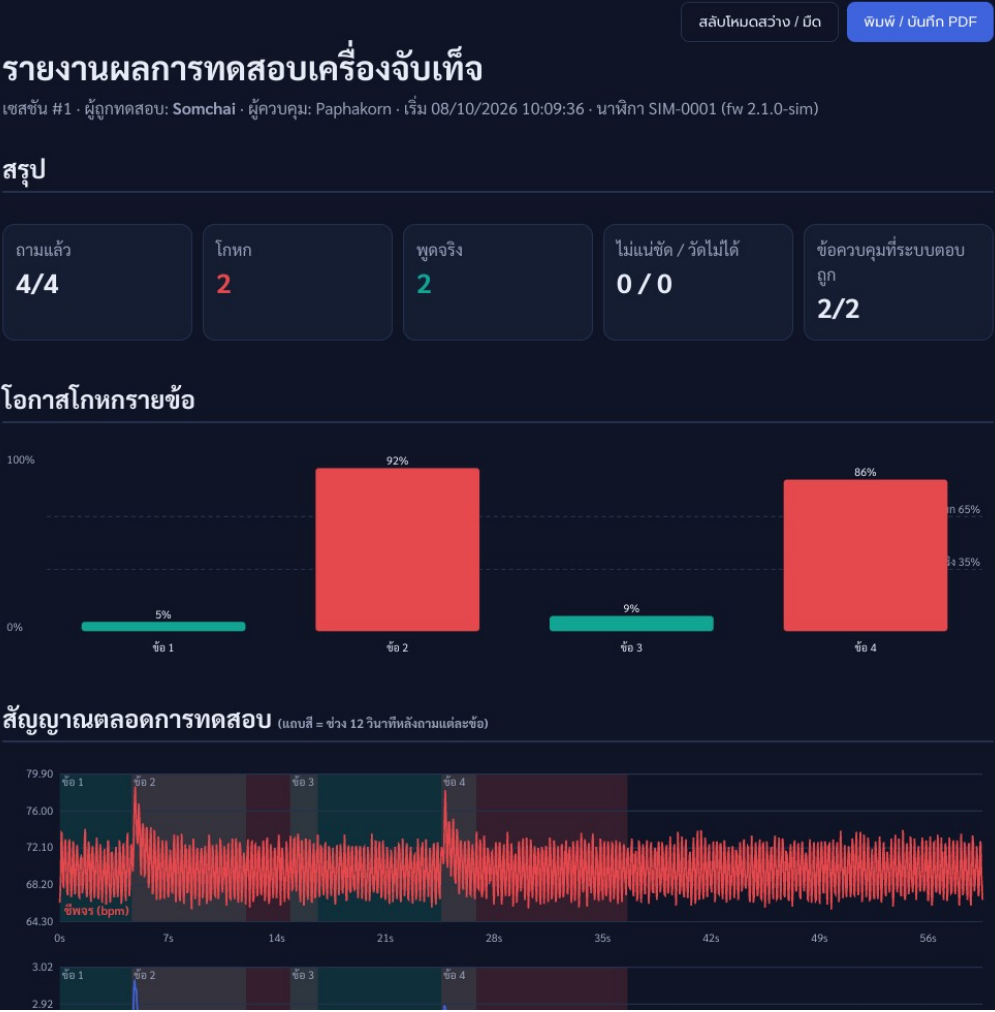


ไม่ต้องมีคอม นาฬิกาเป็นทั้ง backend และ frontend เอง
ไม่ต้องมีเน็ต นาฬิกาปล่อย WiFi เอง (SoftAP)
ครบในหน้าเดียว เลือกรูปแบบ · ผลล่าสุด · ควบคุม · ค่าสด + กราฟ · โมเดล AI · ตั้งค่าพลังงาน
ดาวน์โหลด CSV ข้อมูลเทรนในนาฬิกา
อัปโหลด model.json ติดตั้ง AI เข้านาฬิกาจากมือถือ

เชื่อมต่อแบบเสียบสาย vs WiFi

หัวข้อ	USB (สาย Type-C)	WiFi (SoftAP)
ใช้ทำอะไร	อัปเดตเฟิร์มแวร์ · Serial Monitor พิมพ์คำสั่ง	หน้าเว็บ · Studio · เก็บข้อมูล · OTA
ข้อมูลสด	ข้อความ/raw ทาง Serial	UDP 5 ครั้ง/วินาที + HTTP
อุปกรณ์ที่ใช้ได้	คอมเครื่องเดียว	มือถือ + คอม พร้อมกันได้ (สูงสุด 3 เครื่อง)
ความปลอดภัยตอนวัด GSR	ไม่ควรวัดกับคน (ผิวต่อกับไฟบ้านผ่านคอม)	ใช้แบตเตอรี่ แยกจากไฟบ้าน
ข้อดี	เสถียร ไม่หลุด ดีบักง่าย	อิสระ ใส่ข้อมือได้จริง
ข้อจำกัด	ติดสาย	ระยะจำกัดด้วยเสาอากาศเล็กและกำลังส่งที่ลดไว้

รายงานผลและฐานข้อมูล



SQLite (studio.db) sessions · questions · results · samples · events · training_runs · training_questions
VIEW สำหรับเปิดดูใน DBeaver v_sessions_overview · v_question_results · v_training_questions
Data Dictionary อธิบายทุกคอลัมน์ของ CSV และตาราง (เปิดจาก Studio ได้)

05

AI & Backend

เก็บข้อมูลลง CSV · Label จริง/โกหก · เทรนและใช้บน
นาฬิกา

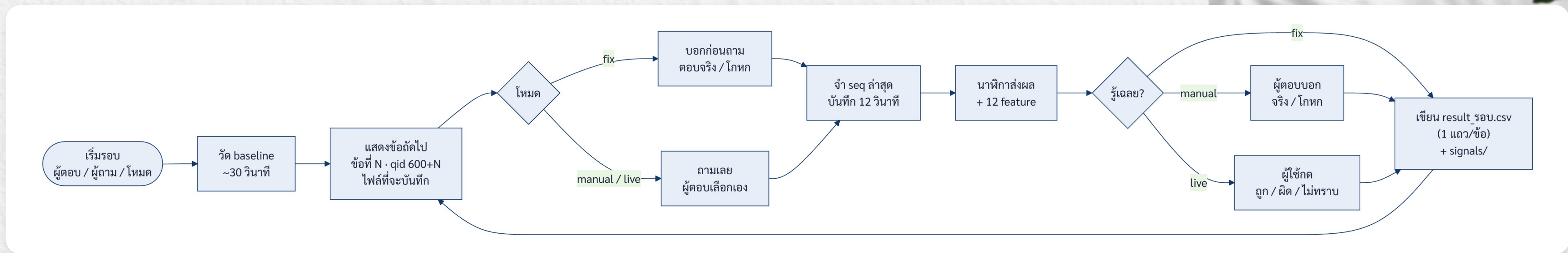


Backend: ไฟล์ .bat ทำอะไรบ้าง

ไฟล์	ทำอะไร	ใช้ตอนไหน
install.bat	ติดตั้งไลบรารี Python (FastAPI, uvicorn, websockets ...)	ครั้งแรกครั้งเดียว (ต้องมีเน็ต)
run_studio.bat	เปิด Polygraph Studio เชื่อมนาฬิกาจริง 192.168.4.1	ใช้งานจริง / นำเสนอ
run_demo_simulator.bat	เปิดนาฬิกาจำลอง + Studio + หน้าเว็บนาฬิกา	ซ้อม/ทดสอบโดยไม่มีบอร์ด
collect_data.bat	เก็บข้อมูลเทรนแบบทีละขั้น (fix/manual) ลง data/result_<วันเวลา>.csv	เก็บข้อมูลจากคอมโดยไม่เปิด Studio
train_ai.bat	เทรน AI จาก data\result_*.csv เท่านั้น → model.json (+ ส่งเข้านาฬิกาถ้าต่ออยู่)	หลังเก็บข้อมูล
train_ai_upload.bat	ส่ง model.json ที่มีอยู่เข้านาฬิกา (ไม่เทรนใหม่)	ทางเลือก — Studio ส่งให้อัตโนมัติอยู่แล้ว

โค้ด: backend/ (FastAPI) · frontend/ (HTML/JS) · ml/ (เก็บข้อมูล + เทรน, ใช้แต่ไลบรารีมาตรฐาน) · tools/virtual_watch.py (นาฬิกาจำลอง)

เก็บข้อมูลอย่างไร และทำไมเลือก CSV



เปิดด้วย Excel ได้เลย

ตรวจสอบข้อมูลด้วยตาได้ทันที

เขียนต่อท้ายทีละแถว

ไฟล์ดับ/ปิดโปรแกรม ข้อมูลก่อนหน้า
ยังอยู่

ทุกแถวบอกที่มา

question_no · watch_qid · label ·
phase

ไม่สร้างไฟล์เปล่า

สร้างเมื่อเริ่มข้อแรก

ไฟล์	1 แถว =	ใช้ทำอะไร
signals/signals_<วันเวลา>.csv	ค่าเซนเซอร์ 1 ครั้ง (5 แถว/วินาที)	กราฟ/วิเคราะห์ ว่าแต่ละข้อสัญญาณเป็นอย่างไร
result_<วันเวลา>.csv	1 ข้อ (เฉลย + คำตัดสิน + 12 feature)	ไฟล์เดียวที่ใช้เทรน AI + เทียบเฉลยกับคำตัดสิน
model.json	โมเดลล่าสุด (w, b, mean, scale)	Studio ส่งเข้านาฬิกาอัตโนมัติ

ตัวอย่างข้อมูลที่ได้ (result_<วันเวลา>.csv)

question_no	watch_qid	mode	label	verdict	p_lie	decided_by	d_hr	z_hr	z_gsr
1	600	fix	truth	truth	0.061	rules	3.99	0.15	0.24
2	601	fix	lie	lie	0.972	rules	10.64	4.58	7.08
3	602	fix	lie	lie	0.840	rules_calibrated	9.51	3.83	5.35
4	603	fix	truth	truth	0.054	rules_calibrated	3.81	0.03	-0.26

label

เฉลี่ย: truth = ตอบจริง, lie = โกหก

verdict / p_lie

คำตัดสินและโอกาสโกหกจากนาฬิกา

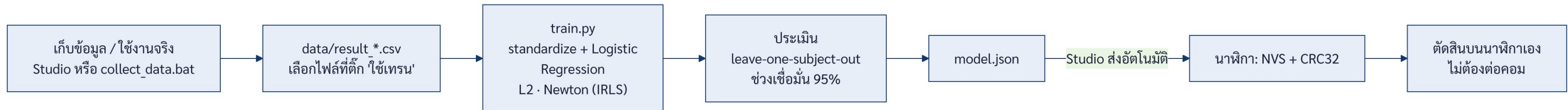
decided_by

rules = สูตร, rules_calibrated = สูตรปรับจากข้อความ, ai_model = AI

d_hr / z_hr

ชีพจรเพิ่มกี่ bpm / มากกว่าตอนพักกี่เท่า

เทรน AI อย่างไร



โมเดลและการประเมิน

Input 12 ค่า z_gsr z_hr z_amp z_trm z_tmp, d_* 5 ตัว, gsr_ok, ppg_ok

Logistic Regression + L2 $p(\text{โกหก}) = \text{sigmoid}(b + \sum w_i \cdot x_i)$ หาค่า w ด้วย Newton (IRLS)

Leave-one-subject-out เทรนด้วยคนอื่น ทดสอบกับคนใหม่ (≥ 3 คน)

รายงานผล ความแม่นยำ + ช่วงเชื่อมั่น 95%, sensitivity, specificity, เทียบกับสูตรเดิม

ml/train.py

```
# 1) อ่านเฉพาะ data/result_*.csv ที่ติดใช้เทรน
ds = build_dataset(df.training_files(DATA), feats)
# 2) ประเมินแบบไม่แอบดูข้อสอบ
method, p, _ = pm.cross_validate(ds, feats)
print(pm.metrics(ds.y, p))
# 3) เทรนจริงด้วยข้อมูลทั้งหมด
model = pm.Model.fit(ds.X, ds.y, feats, lam)
# 4) บันทึก model.json -> Studio ส่งเข้านาฬิกาเอง
save_model(model.to_json(**info))
# model_sync.py: เทียบกับนาฬิกาทุก 4 วิ แล้วส่งให้
```

AI ทำงานบนนาฬิกาเอง

```
src/lie/ml_model.cpp

float predict(const Model& m, const Result& r) {
    float x[FEAT_COUNT];
    extract(r, x);                // 12 feature
    double z = m.b;
    for (int i = 0; i < m.n; i++)
        z += m.w[i] * (x[m.idx[i]] - m.mean[i])
                / m.scale[i];
    return 1.0 / (1.0 + exp(-z)); // sigmoid
}
```

	RandomForest (โค้ดเดิม)	Logistic Regression (ที่ใช้)
รันบน ESP32-C3	ยาก (ต้นไม้ 100 ต้น)	ได้ — คูณ-บวก 12 ครั้ง
ข้อมูลหลักสืบข้อ	overfit ง่าย	เหมาะกว่า (มี L2)
อธิบายผล	ยาก	ดูน้ำหนัก w ได้ว่าสัญญาณไหนสำคัญ
เทียบความแม่นยำ	-	train.py --compare-rf

- ไม่มีโมเดล นาฬิกาใช้สูตรถ่วงน้ำหนัก (ปรับจากข้อควบคุมได้)
- มีโมเดล ทุกผลบอก "ตัดสินใจด้วยโมเดล AI" ทั้งบนเว็บ, Studio และไฟล์ CSV
- ปลอดภัย ตรวจสอบขนาด/ค่า/CRC32 ก่อนติดตั้ง ลบโมเดลได้ตลอด (model clear)

ฟีเจอร์เพิ่มเติม

1

Serial Console

พิมพ์ help · status · i2c · mode · data · sleep · boots · tasks · flash

2

OTA + Rollback

อัปเดตเฟิร์มแวร์ผ่าน WiFi ถ้าเฟิร์มแวร์ใหม่ล้ม ย้อนกลับเอง

3

ตั้งค่า Sleep ง่าย ๆ

ค่าเริ่มต้นไม่หลับ ตั้งได้จากเว็บ / Serial / ปุ่ม

4

นาฬิกาจำลอง

ซ็อนนำเสนอและทดสอบซอฟต์แวร์ได้โดยไม่มีบอร์ด

5

ไฟ LED 5สี

บอกสาเหตุบูต / WiFi / แบตต่ำ โดยไม่ต้องมีจอ

6

ประวัติการรีเซ็ต

นับ watchdog / ไฟตก / panic เก็บในแฟลช

7

ใช้กับโค้ดเดิมได้

ส่ง "hello" เฉย ๆ ได้ CSV รูปแบบเดิม

8

Unit test

ทดสอบ DSP + LieEngine บนคอม และเทียบผล C++ กับ Python

06

Limitations & Solutions

ปัญหาที่เจอ · สาเหตุ · วิธีแก้



ปัญหาที่พบและวิธีแก้

ปัญหา	สาเหตุ (ที่พบ / ที่วิเคราะห์)	วิธีแก้
ใช้แบตเตอรี่ WiFi ไม่ขึ้น / เสียบ USB แล้วเซนเซอร์ไม่ติด	สันนิษฐาน: ขา 3V3 ที่ซ็อกเก็ตต่อไม่ถึงราง 3V3 และ HT7333 จ่ายได้ 250 mA แต่ WiFi พีกถึง 335 mA (ตรวจด้วยมัลติมิเตอร์)	บัดกรีจุดต่อใหม่ · ตัวเก็บประจุ 470 μ F · ลดกำลังส่ง WiFi · ไฟ LED + คำสั่ง boots บอกลไฟตก
คนไม่ได้มีเหงื่อตลอด	GSR หายเมื่อผิวแห้ง ระบบเดิมพึ่ง GSR มากเกิน	ลดน้ำหนัก GSR เหลือ 20% · ตัดสินจากสัญญาณอื่นได้
Serial พิมพ์แล้วเงียบ	ตั้ง timeout การส่งเป็น 0 ทำให้ข้อความยาวถูกทิ้ง	ตั้ง 50 ms + ส่งตัวอักษรกลับ (echo)
เซนเซอร์ความเร่งเป็นรูลื่นเลียนแบบ	WHO_AM_I = 0x70 (MPU6500)	driver อ่านรูลื่นชิปแล้วตั้งค่าให้ตรง
บัส I2C ค้างเป็นบางครั้ง	สายหลวม/สัญญาณรบกวน	กู๊บัสอัตโนมัติ (ส่ง clock 9 ลูก) + watchdog

ข้อจำกัดและแนวทางต่อยอด

ข้อจำกัด

วัดความตื่นตัว ไม่ใช้การโกหกโดยตรง คนพูดจริงที่ตื่น
ตื่นอาจถูกตัดสินว่าโกหก

โกหกตามคำสั่ง $\neq$ โกหกจริง ความแม่นยำใช้ได้กับ
สถานการณ์ทดลองเท่านั้น

ข้อมูลยังน้อย ต้องเก็บหลายคน รายงานช่วงเชื่อมั่นทุก
ครั้ง

ESP32-C3 แขนเดียว RAM 400 KB ต้องใช้โมเดลเบา
ระยะ WiFi เสืออากาศเล็ก + ลดกำลังส่งเพื่อลดกระแส

แนวทางต่อยอด

- เปลี่ยน LDO เป็นรุ่นจ่าย ≥ 500 mA เช่น AP2112K-3.3
- เก็บข้อมูลหลายคนขึ้น แล้วเทียบโมเดลแบบอื่น
- เพิ่ม Bluetooth LE สำหรับมือถือโดยไม่ต้องสลับ WiFi
- ทำกล่อง/สายรัดที่กดเซนเซอร์แบบพิวสมาเสมอ
- ใช้เพื่อการศึกษาเท่านั้น ไม่ใช่ตัดสินคนจริง

DEMO

- 1 ใส่หน้ากาก ดูค่าสดและกราฟบนจอ
- 2 วัด baseline 30 วินาที
- 3 ถามข้อความคลุม: ตอบจริง / สั่งให้โกหก
- 4 ถามคำถามจริง → ผลจริง/โกหก + เหตุผล
- 5 ลองให้ผ่าน GSR ไม่แตะผิวหนัง → ยังตัดสินใจได้
- 6 สาธิต watchdog → รีเซ็ตแล้วบอกสาเหตุ





Thank you

ขอบคุณครับ / ค่ะ — ยินดีตอบคำถาม

อัจฉรา ตั้งดี 6730300655 · ปภากร จันทร์ดี 6730300809

03603323 Introduction to Embedded Systems · วิศวกรรมคอมพิวเตอร์ มก. ศรีราชา